

1) Key features of JavaScript

Interpreted and lightweight scripting language
Object-Oriented (prototype-based)
Event-driven programming support
Platform independent (runs on browsers)
Case-sensitive language
Supports dynamic typing
Single-threaded with asynchronous capabilities (via callbacks, promises, async/await)

2) JavaScript is Lightweight Scripting Language – Justify

It doesn't need heavy compilers; runs inside the browser.
Uses fewer resources compared to traditional programming languages.
Has minimal syntax rules and simple structure.
Executes directly within HTML without extra tools.

3) JavaScript is Single-threaded – Justify

JavaScript executes one task at a time (single call stack).
It follows the "event loop" model → manages tasks asynchronously without creating new threads.
This prevents race conditions and makes JS simple to use in browsers.

4) Is JavaScript Compiled or Interpreted or both?

Both.
Traditionally, JavaScript was interpreted by browsers.
Modern JS engines (like V8 in Chrome) compile code into machine code (JIT – Just In Time Compilation) before execution.
So, JavaScript is interpreted + JIT compiled.

5) Applications of JavaScript

Web development (front-end: interactive websites)
Server-side development (Node.js)
Mobile app development (React Native, Ionic)
Desktop apps (Electron.js)
Game development
Internet of Things (IoT)

6) Functionality of JavaScript Engine

Parses and executes JavaScript code.
Components:
Parser → Reads code and checks syntax.
Interpreter/Compiler (JIT) → Converts JS to machine code.
Memory Heap & Call Stack → Manages memory and execution order.
Garbage Collector → Frees unused memory.

7) Key Panels in DevTools for JavaScript

Console → Execute JS, debug errors.
Sources → View, edit, and debug code.
Network → Monitor API requests.
Elements → Inspect and modify HTML/CSS.
Performance → Check speed and performance issues.
Application → Manage storage (cookies, local storage).

8) Difference between Java and JavaScript

Feature	Java	JavaScript
Type	Object-Oriented Programming Language	Scripting Language
Compilation	Compiled	Interpreted + JIT compiled
Platform	Standalone apps, backend	Web development, server (Node.js)
Syntax	Strict, class-based	Flexible, prototype-based
Usage	Mobile, desktop, enterprise apps	Web apps, interactive UI

9) How to Link JavaScript in an HTML File

Inline:
<button onclick="alert('Hello!')">Click Me</button>

Internal (inside <script>):

```
<script>
  console.log("Hello World");
</script>
```

External (recommended):

```
<script src="script.js"></script>
```

10) Different Ways to Declare Variables

var → Function scoped, older (avoid mostly).

let → Block scoped, can be reassigned.

const → Block scoped, cannot be reassigned.

11) Different Data Types in JavaScript

Primitive: string, number, boolean, null, undefined, symbol, bigint

Non-primitive: object, array, function

12) Different Operators in JS

Arithmetic: + - * / % **

Assignment: = += -= *= /=

Comparison: == === != !== > < >= <=

Logical: && || !

Bitwise: & | ^ ~ << >>

Ternary: condition ? value1 : value2

13) Decision Making Statements in JS

if statement

if...else

if...else if...else

switch statement

14) Loops in JS

for loop → fixed repetitions

while loop → runs until condition false

do...while loop → runs at least once

for...in → loops through object properties

for...of → loops through iterable objects (arrays, strings)

15) Explain the use of the ternary operator with example.

It is a shorthand for if-else.

Syntax: condition ? value1 : value2

Example:

let age = 20;

let result = (age >= 18) ? "Adult" : "Minor";

16) Write a note on for...in and for...of loops.

- **for...in:** Iterates over object properties.

for (let key in obj) { console.log(key); }

- **for...of:** Iterates over array or string values.

for (let val of arr) { console.log(val); }

17) Discuss different looping structures in JavaScript with examples.

- **for loop:** for(let i=0;i<5;i++){console.log(i);}
- **while loop:** while(i<5){ console.log(i); i++; }
- **do-while loop:** Runs at least once.
- **for...in:** Iterates over object keys.
- **for...of:** Iterates over iterable values.

18. Explain control flow statements (if, else, switch) with examples.

- **if-else:** Executes code based on condition.

if (marks >= 40) console.log("Pass"); else console.log("Fail");

- **else if:** Multiple conditions.

if (n>0) console.log("Positive"); else if(n<0) console.log("Negative"); else console.log("Zero");

- **switch:** Checks a variable against multiple cases.

switch(day){ case 1: console.log("Mon"); break; default: console.log("Other"); }

19) Differentiate between function declaration and function expression.

- **Declaration:** Named function, hoisted.

function greet(){ return "Hello"; }

- **Expression:** Assigned to variable, not hoisted.

let greet = function(){ return "Hello"; };

20). What are arrow functions? Give an example.

A shorter way of writing functions using =>.

let sum = (a,b) => a+b;

21) Define arrays in JavaScript. Write any two array methods.

Array is a collection of ordered elements.

Example:

let arr = [1,2,3];

arr.push(4); // adds at end

arr.pop(); // removes last element

22) Define objects in JavaScript with example.

Objects are key-value pairs.

```
let student = { name:"Ali", age:20 };  
console.log(student.name);
```

23) What is closure in JavaScript? Explain briefly.

A closure is a function that remembers variables from its outer scope even after the outer function has finished.

```
function outer(){ let x=10; return function(){ return x; } }  
let inner=outer(); console.log(inner()); // 10
```

24) Explain functions in JavaScript (declaration, expression, arrow functions) with examples.

- **Declaration:** `function greet(){return "Hi";}`
- **Expression:** `let greet=function(){return "Hi";}`
- **Arrow:** `let greet=()=> "Hi";`
Functions may take parameters and return values.

25) Discuss important array methods (push, pop, map, filter, reduce) with examples.

- `push()` → adds element at end
- `pop()` → removes last element
- `map()` → returns new array after applying function
- `filter()` → returns array with elements matching condition
- `reduce()` → reduces array to single value (like sum)

26) Explain scope in JavaScript (global, local, block) with examples.

- **Global Scope:** Accessible everywhere
- **Local Scope:** Defined inside function
- **Block Scope:** Variables with `let/const` inside `{}`

```
let a=10; // global  
function test(){ let b=20; } // local  
if(true){ let c=30; } // block
```

Q4. Explain objects in JavaScript along with destructuring example.

Objects store data in key-value pairs.

```
let car={brand:"Honda",year:2020};  
let {brand,year}=car; // destructuring
```